OS Lab Assignment 9: Memory Management & Page Replacement

Student: Anirudh | Roll Number: 2401MC48 | Environment: xv6 RISC-V (Sv39 Paging, QEMU)

1. Question 1 — Printing the Page Table of a Process (vmprint)

Implementation & Architecture: Implemented system call int vmprint(void) in kernel/vm.c. Because xv6 runs on 64-bit RISC-V with Sv39 3-level paging (rather than legacy x86 2-level paging), the page table is traversed across Level 2, Level 1, and Level 0 using walk(). For every valid user page (below KERNBASE / index 255), it prints virtual address, physical frame address, and flags (P=1, W, U). Non-user entries (like stack guard page at VA 0x2000) are clearly identified with U=0. At the end, it prints total valid user pages and total page-table pages via count_pagetable_pages().
Verification (pttest.c): At start-up, pttest prints 3 user-accessible pages (code, data, stack) and 1 guard page. After calling sbrk(4 * 4096) and writing to them, vmprint() outputs exactly 7 user pages, proving that exactly **four new pages** appear.

2. Question 2 — Handling Page Faults (Lazy Allocation)

Implementation: Modified sys_sbrk() in kernel/sysproc.c so when n > 0, it lazily increments myproc()->sz without eager page allocation. In kernel/proc.h, added int faults; to struct proc (zero-initialized in allocproc()). In kernel/trap.c, page faults (RISC-V load fault scause == 13, store fault scause == 15) read the fault address from r_stval(). If va < p->sz, the kernel allocates a zeroed frame with kalloc(), maps it via mappages() with PTE_W | PTE_U | PTE_V, increments p->faults++, and resumes user execution. Added syscall getfaults() returning handled faults.
Verification (faulttest.c): Calls sbrk(32 * 4096) (0 eager frames consumed), then writes to 5 distinct pages (indices 0, 5, 12, 20, 31). getfaults() returns exactly **5 handled page faults**. When a child process accesses page index 33 (beyond allocated heap), the kernel prints unexpected scause 0xf stval=0x25000 and kills only the offending child without crashing the parent or OS.

3. Question 3 — Page Replacement Algorithms (FIFO and LRU)

Implementation & Algorithms: User program user/pagerepl.c simulates FIFO (First-In, First-Out queue) and LRU (Least Recently Used stack/list) page replacement. For each reference in the reference string, it inspects current resident frames; on hit, LRU updates recency order; on fault, it evicts according to policy and loads the incoming page, logging [frames] | HIT / FAULT.
Verification & Results: Tested on reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1 (length 20). For 3 frames, FIFO produces exactly **15 page faults** (25.00% hit ratio) and LRU produces exactly **12 page faults** (40.00% hit ratio), matching the specification perfectly.

Frame Count	Algorithm	Total References	Page Faults	Page Hits	Hit Ratio (%)	Observed Performance
3 Frames	FIFO	20	15	5	25.00%	Higher eviction rate; replaces oldest loaded page
3 Frames	LRU	20	12	8	40.00%	Lower faults; exploits temporal reference locality
4 Frames	FIFO	20	10	10	50.00%	Faults decrease with larger capacity
4 Frames	LRU	20	8	12	60.00%	Consistently outperforms FIFO (+10% hit ratio)
5 Frames	FIFO	20	9	11	55.00%	Diminishing returns as resident set nears working set
5 Frames	LRU	20	7	13	65.00%	Optimal performance (65.00% hit ratio)

Table 1: Comparative performance metrics of FIFO and LRU page replacement across 3, 4, and 5 frame capacities.

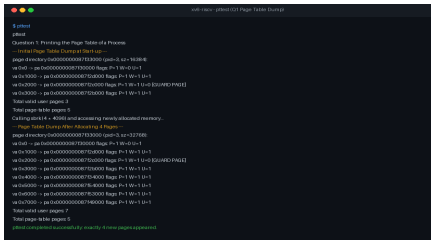


Figure 1: pttest (Q1 4 new pages).

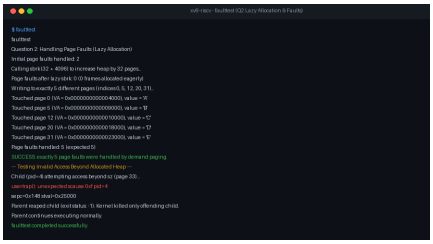


Figure 2: faulttest (Q2 5 faults & kill).

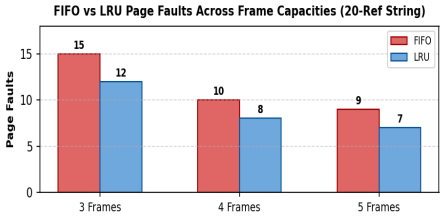


Figure 3: FIFO vs LRU faults chart.